Comparing Time

Ruby, not content with a simple date-and-time system, has three separate classes that manage date and time data. The Time class is a thin wrapper around the same Unix C library that pretty much every language exposes. (Motto: "Annoying programmers since 1983!") The Ruby-specific classes Date and DateTime are more flexible and have a more coherent API but are slower.

For Rails testing purposes, the relevant points are that ActiveRecord uses Date and DateTime (depending on the specifics of the underlying database column). Comparing a Date to a DateTime instance will always fail, as will trying to add or subtract them. And most of the Rails ActiveSupport methods (such as 5.days.ago) return DateTime. In testing this can lead to a lot of annoying failures, especially when you have a Date column with no time information—which is recommended if the time is not important.

In general, it's a good idea to compare dates and times by converting them using to_s(:db). It avoids the irritating question of object equality and you get more readable tests and error messages. When the exact time of the time object is in question, try to force the issue by using the Rails ActiveSupport methods to_date, to_time, and to_datetime. At worst, this means something like 5.days.ago.to_date.to_s(:db), which may read a touch awkwardly but is a robust test with a decent error message on failure.

Setting Rails Timestamps

One trick when testing dates is to explicitly set your ActiveRecord model's created_at attribute. Normally created_at is a timestamp automatically generated by Rails, and it's often used for the kind of time-based reporting alluded to in the rest of this section. Since it's automatically created at the current time, you can get into some weird situations if the other dates in the test are explicitly set in the past. Even without that complication, you may still need to explicitly set created_at to use the attribute to test time-based features.

You can set created_at in the fixture file, just like any other attribute; specify it in ActiveRecord::create, ActiveRecord::new, or a factory blueprint or just reset with an assignment or update method.

Setting updated_at is trickier. Under normal circumstances, if you try to explicitly set updated_at Rails will automatically reset it on save, which defeats the purpose. To change this behavior, set the class variable for your class with code like Project.record_timestamps = false, using the name of your class as the message receiver sometime before you save the object with a modified update time. After the save, reset things to normal with Project.record_timestamps = true.